

5-2 · Spark 마스터-워커 구축 · 실습 1.5H

Docker Compose로 마스터-워커 클러스터 구축

마스터 1 + 워커 2 를 compose 하나로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① 8080 포트를 비우고 `compose` 파일을 직접 쓴다
- ② `docker compose up -d` 로 마스터 1 + 워커 2 를 띄운다
- ③ 로그와 마스터 UI 에서 워커 등록(ALIVE)을 확인한다
- ④ `--scale` 로 워커를 늘렸다 줄이며 UI 가 따라 바뀌는 것을 본다

RAM 16GB 환경이면 워커를 1개로 — 마스터-워커 구조와 제출 흐름은 같습니다

일곱 단계 — compose 파일 하나로 클러스터를

~/data-eng/week5 폴더에서 시작합니다 — 4주차 컨테이너가 8080 을 쓰고 있으면 먼저 내려 둡니다

1. 포트 비우기	2. compose 작성	3. 기동	4.5. 로그·UI	6.7. 스케일
<pre>docker ps 8080 확인</pre>	<pre>master + worker replicas: 2</pre>	<pre>config → up -d → ps</pre>	<pre>등록 기록 localhost:8080</pre>	<pre>--scale 3 → 2 hostname 비교</pre>

compose 파일 — 서비스 두 개, 핵심 줄 여덟

줄	뜻
image: apache/spark:3.5.3	마스터·워커 같은 이미지
...deploy.master.Master	마스터 — Master 클래스 실행
8080:8080 · 7077:7077	마스터 웹 UI · 접속 포트
...deploy.worker.Worker	워커 — 첫 인자로 마스터 주소
spark://spark-master:7077	마스터 주소 = 서비스 이름
--cores 2 --memory 2g	이 워커가 클러스터에 내놓을 자원
depends_on: spark-master	마스터를 먼저 띄운다
deploy: replicas: 2	워커 2개 (16GB 면 1개)

RAM 16GB 면 replicas: 1 또는 deploy 블록 삭제 — 워커 2 × 2G 에 마스터·드라이버·OS 까지 더하면 빠듯하다

핵심 명령 — 비우고 · 확인하고 · 띄우고 · 로그

포트 비우기	cd ~/lakehouse && docker compose stop
문법 확인	docker compose config
기동	docker compose up -d
상태	docker compose ps
마스터 로그	docker compose logs spark-master
워커 로그	docker compose logs spark-worker

로그에 `grep -i "registering worker"`(마스터) · `"successfully registered"`(워커)를 붙이면 등록 기록만 남는다

기동 뒤 확인 — ps · 로그 · 마스터 UI

1 docker compose ps — 모두 Up
spark-master 1개와 spark-worker 컨테이너가 전부 Up 상태

2 로그 — 등록 기록 두 줄
마스터 「Registering worker ...」 · 워커 「Successfully registered with master」

3 localhost:8080 — Workers 표
워커들이 ALIVE — 이론의 흐름 ②~④ 가 로그와 화면으로 남은 것

워커 2개일 때 UI 수치

Cores 4
Memory 4G

마스터 UI 화면 — 네 곳을 읽는다

1 URL: spark://spark-master:7077

다음 1H 에 잡을 제출할 때 쓸 주소

2 Alive Workers · Cores in use · Memory in use

지금 클러스터가 가진 자원 — 오늘 제출할 잡이 쓸 수 있는 상한

3 Workers 표

워커마다 주소·상태·코어·메모리

4 Running / Completed Applications

잡 이력 — 지금은 비어 있고, 다음 1H 에 채워진다

워커 2개 구성이면 Cores in use 4 · Memory 4.0 GiB 안팎 — 이 숫자가 오늘 제출할 잡이 쓸 수 있는 상한

워커를 늘렸다 줄이기 — 죽는 것도 본다

--scale 로 워커 수를 바꾸며 UI 가 따라오는 것을, 메모리를 넘기면 뜨자마자 죽는 것을 봅니다

3개로	UI 새로고침	과하게(실패)	2개로	hostname
--scale spark-worker=3	Workers 3 Cores 따라 변함	메모리 넘으면 ps 에 Exited	--scale spark-worker=2	마스터·워커 이름이 다르다

같은 `--scale` 인데 — 뜨는 것과 죽는 것

워커 2 → 3 — 정상

UI 의 Workers 수와 Cores in use 가 따라 는다



VS

메모리 초과 — 뜨자마자 죽음

Docker 에 할당된 메모리를 넘으면 컨테이너가 죽는다



이 1.5시간이 끝날 때 확인할 것

- 1 `docker ps` 에 8080 을 쓰는 컨테이너가 없다
- 2 `docker compose ps` 에 `spark-master` 1개와 워커가 모두 Up 이다
- 3 워커 로그에 `Successfully registered with master` 가 있다
- 4 마스터 UI Workers 표에 워커가 ALIVE 로 보인다
- 5 `--scale` 뒤 Workers 수와 Cores in use 가 따라 바뀐다
- 6 마스터와 워커들의 hostname 이 서로 다르다

5-2 · 실습 1.5H 마무리

다음은 spark-submit 실습 (1H)

마스터 컨테이너 안의 spark-submit 으로 예제와 직접 쓴 스크립트를 제출하고, 마스터 UI 에서 실행을 추적합니다

✓ pi.py 예제 제출 — Pi is roughly 3.14

✓ first_job.py 제출 — master 값 확인

✓ --total-executor-cores 로 자원 제한

✓ 메모리를 과하게 요구해 WAITING 재현

가져갈 것

클러스터 Up 상태 · 마스터 UI 8080